

SIJ.JAVA

```

/* Serial Interfacing In Java application */
package SIJ.classes;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JScrollPane;
import javax.swing.JOptionPane;
import javax.swing.ImageIcon;
import java.awt.Dimension;
import java.awt.Toolkit;
import java.awt.TrayIcon;
import java.awt.BorderLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.awt.event.JToggleButton;

public class SIJ implements ActionListener {
    public JFrame frame;
    private JTextArea rectext, sndtext;
    private JTextField sndfld;
    private JTextCreator tray;
    private JButton sndbut, clrbut;
    private JToggleButton connectbut;
    private JCheckBox clrchk;
    private JComboBox serialport, budrate,
    , databit, parity, stopbit;
    private SendData sdta;
    private ClassLoader cl;
    public SIJ() {
        String[] budval = { "110", "300", "600",
        "1200", "2400", "4800", "9600", "14400",
        "19200", "28800", "38400", "56000" };
        String[] dataval = { "5", "6", "7",
        "8" };
        String[] parityval = { "NONE", "ODD", "EVEN", "MARK", "SPACE" };
        String[] stopval = { "1", "2",
        "1.5" };
        frame = new JFrame("Serial Port Interfacing");
        rectext = new JTextArea();
        sndtext = new JTextArea();
        sndfld = new JTextField();
        sndbut = new JButton("Send");
        clrbut = new JButton("Clear");
        connectbut = new JToggleButton("Connect");
        clrchk = new JCheckBox("Clear data");
        sdta = new SendData(frame, rectext, sndtext);
        serialport = new JComboBox(sdta.getPortIds());
        budrate = new JComboBox(budval);
        databit = new JComboBox(dataval);
        parity = new JComboBox(parityval);
        stopbit = new JComboBox(stopval);
        sdta.setPortId((String) serialport.getSelectedItem());
        new MenuBarCreator(frame);
        cl = getClass().getClassLoader();
        tray = new TrayCreator(frame, "Serial Port Interfacing");
        go();
    }
    private void go() {
        JPanel frmpan = new JPanel();
        JPanel serialpan = new JPanel();
        JScrollPane recscr = new JScrollPane(rectext);
        JScrollPane sndscr = new JScrollPane(sndtext);
        JLabel reclab = new JLabel("Received Data:");
        JLabel sndlab = new JLabel("Send Data:");
        JLabel budlab = new JLabel("BaudRate:");
        JLabel datablab = new JLabel("DataBit:");
        JLabel paritylab = new JLabel("Parity:");
        JLabel stoplab = new JLabel("Stopbits:");
        Dimension scrn = Toolkit.getDefaultToolkit().getScreenSize();
        frame.setBounds(scrn.width/2-200, scrn.height/2-130, 480, 275);
        frame.setIconImage(new ImageIcon(cl.getResource("pic/EG_Icon/EG_Icon.jpg")).getImage());
        frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        frame.setResizable(false);
        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                frame.setVisible(false);
                tray.getTrayIcon().displayMessage("Serial Port Interfacing", "Program is running", TrayIcon.MessageType.INFO);
            }
        });
        frame.getRootPane().setDefaultButton(sndbut);
        recscr.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        recscr.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        sndscr.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        sndscr.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        rectext.setEditable(false);
        sndtext.setEditable(false);
        clrchk.setSelected(true);
        sndbut.setEnabled(false);
        budrate.setMaximumRowCount(5);
        budrate.setSelectedIndex(6);
        databit.setSelectedIndex(3);
        reclab.setBounds(10, 10, 85, 20);
        sndlab.setBounds(180, 10, 70, 20);
        recscr.setBounds(10, 35, 150, 125);
        sndscr.setBounds(180, 35, 150, 125);
        sndfld.setBounds(250, 10, 80, 20);
        budrate.setBounds(353, 10, 98, 20);
        clrbut.setBounds(353, 45, 98, 20);
        connectbut.setBounds(353, 80, 98, 20);
        serialport.setBounds(353, 115, 98, 20);
        clrchk.setBounds(360, 150, 90, 20);
        sndbut.setMnemonic('S');
        clrbut.setMnemonic('C');
        frmpan.setLayout(null);
        frmpan.add(reclab);
        frmpan.add(recscr);
        frmpan.add(sndscr);
        frmpan.add(sndfld);
        frmpan.add(budrate);
        frmpan.add(sndbut);
        frmpan.add(clrchk);
        frmpan.add(connectbut);
        frmpan.add(serialport);
        frmpan.add(clrchk);
        serialpan.add(budlab);
        serialpan.add(datablab);
        serialpan.add(paritylab);
        serialpan.add(stoplab);
        connectbut.addActionListener(this);
        clrbut.addActionListener(this);
        serialport.addActionListener(this);
        frame.getContentPane().add(frmpan, BorderLayout.CENTER);
        frame.getContentPane().add(serialpan, BorderLayout.NORTH);
        frame.getContentPane().add(new JLabel("Developed by Muhammad Ajmal P"), BorderLayout.SOUTH);
        frame.setVisible(true);
        sndfld.requestFocus();
    }
    public static void main(String[] args) {
        SIJ sij = new SIJ();
        private void setComboBox(boolean chs) {
            budrate.setEnabled(chs);
            databit.setEnabled(chs);
            stopbit.setEnabled(chs);
            parity.setEnabled(chs);
            serialport.setEnabled(chs);
        }
        public void actionPerformed(ActionEvent ae) {
            if(ae.getSource() == sndbut) try {
                int recval = Integer.parseInt(sndfld.getText());
                if(recval >= 0 && recval <= 255) sdta.sendValue(recval);
                else JOptionPane.showMessageDialog(frame, "Invalid Input! Enter a value between 0 - 255", "Error", JOptionPane.ERROR_MESSAGE, new ImageIcon(cl.getResource("pic/SIJ/err.png")));
            } catch (Exception e) {
                JOptionPane.showMessageDialog(frame, "Invalid Input! Enter a value between 0 - 255", "Error", JOptionPane.ERROR_MESSAGE, new ImageIcon(cl.getResource("pic/SIJ/err.png")));
            }
            finally {
                if(clrchk.isSelected()) sndfld.setText("");
            }
            else if(ae.getSource() == clrbut) {
                rectext.setText("");
                sndtext.setText("");
            }
            else if(ae.getSource() == connectbut) {
                if(connectbut.isSelected()) {
                    sdta.setPortData(budrate.getSelectedItem().toString(), databit.getSelectedItem().toString(), parity.getSelectedIndex(), stopbit.getSelectedIndex(), connectbut.setText("Disconnect"), sdta.setConnection(true), setComboBox(false), sndbut.setEnabled(true);
                }
                else {
                    connectbut.setText("Connect");
                    sdta.setConnection(false);
                    setComboBox(true);
                    sndbut.setEnabled(false);
                }
            }
            sndfld.requestFocus();
        }
        class comboBoxListener implements ItemListener {
            public void itemStateChanged(ItemEvent ie) {
                if(ie.getSource() == serialport && ie.getStateChange() == ItemEvent.SELECTED) {
                    sdta.setPortId(serialport.getSelectedItem().toString());
                    System.out.println("COM port changed - " + serialport.getSelectedItem().toString());
                }
            }
        }
    }
}

```